Gottfried Wilhelm Leibniz Universität Hannover
Wirtschaftswissenschaftliche Fakultät
Institut für Produktionswirtschaft

Crane Scheduling at Cargo Ports – A Heuristic Solution Approach

Bachelor's Thesis

by Xuan Hoang Nguyen

Examiner: Prof. Dr. Stefan Helber
Student ID number: 10053004

Date of issue: 17.12.2025
Submission date: 17.03.2026

Contents

List of Figures iii

List of Tables iii

List of Algorithms iii

List of Source Codes iii

List of Abbreviations iii

List of Symbols iv

1 Introduction 1

2 The quay crane scheduling problem 2
- 2.1 Operational description and economic implications 2
- 2.2 Review of existing research 4

3 Mathematical Formulation 6
- 3.1 Modeling assumptions 6
- 3.2 Notation 7
- 3.3 A mathematical formulation 8
  - 3.3.1 Objective function 9
  - 3.3.2 Makespan definition 10
  - 3.3.3 Assignment and routing constraints 10
  - 3.3.4 Temporal feasibility constraints 11
  - 3.3.5 Precedence constraints 11
  - 3.3.6 Interference and non-crossing constraints 12
  - 3.3.7 Crane completion and availability constraints 13
  - 3.3.8 Domain restrictions 14

4 Numerical experiments 14
- 4.1 Technical configuration 14
- 4.2 Experimental design 14
- 4.3 Instance generation and parameterization 15
- 4.4 Results 16
  - 4.4.1 Task scaling experiment 16
  - 4.4.2 Capacity test 17
  - 4.4.3 Time limit experiment 18

5 Heuristic Solution Approach **20**
5.1 Design Principles 20
5.2 Construction Phase 21
5.3 Improvement Phase 22
5.4 Illustrative Numerical Example 23

6 Computational Experiments **26**
6.1 Benchmark Design 26
6.2 Heuristic Parameter Settings 27

A Python code **30**

List of Figures

1 A figure of quay cranes working on a vessel. 2
2 An example of a single bay stowage plan. 3
3 Optimality rate vs number of tasks. 18
4 Optimality gap as a function of the time limit (five seeds). 19
5 Evolution of primal objective and best bound over increasing time limits. 19

List of Tables

1 Summary of notation 8
2 Summary of instance parameters and experimental settings. 16
3 Results of the task scaling experiment. 17
4 Input data for the numerical example 23
5 Timed schedule obtained by greedy construction 25
6 Timed schedule after an improving intra-insert move 26
7 Parameter settings of the tabu search heuristic 28

List of Algorithms

1 Greedy & Tabu search 21

List of Source Codes

1 MILP model 30
2 MILP solver 33
3 Instance generator and problem data 35
4 MILP benchmark 36

List of Abbreviations

QCSP Quay Crane Scheduling Problem

MILP Mixed-integer Linear Program

List of Symbols

d_{ij}	Parameter representing the distance from location i to location j
$i \in \mathcal{I}$	Customer location i in the set $\mathcal{I}$
Y_{im}	Binary variable equal to 1 if location i is included in tour m

1 Introduction

Cargo ports constitute critical nodes in global supply chains and are essential for the functioning of international trade. A substantial share of worldwide cargo flows is handled via maritime transport, which places high operational demands on container terminals. From a business perspective, terminal operators face continuous pressure to increase throughput while maintaining reliability and cost efficiency. Operational inefficiencies become visible in the form of vessel delays, contractual penalties, and increased operating costs. Consequently, the effective planning of terminal resources is of central economic importance.

Within container terminals, quay cranes perform the transshipment of containers between vessels and the landside transport system. These cranes represent a key and limited resource. Their size, acquisition cost, and technical constraints require careful planning and coordinated deployment. Inefficient crane allocation directly affects vessel turnaround times and, therefore, overall terminal performance. The operational challenge of assigning and sequencing quay cranes to service vessels can be formalized as the Quay Crane Scheduling Problem (QCSP).

The QCSP belongs to the class of combinatorial optimization problems. Such problems can be addressed using exact solution methods, which aim to determine optimal solutions. However, exact approaches may become computationally demanding for realistically sized instances. An alternative approach consists of heuristic methods, which seek high-quality solutions within limited computation times without guaranteeing optimality.

Against this background, this thesis investigates the following research question:

How do heuristic methods compare to an exact formulation in terms of solution quality, computational efficiency, and scalability for the QCSP?

The structure of this thesis is as follows. Chapter 2 provides a formal problem description and outlines the scope of the analysis. It further reviews relevant approaches from the literature and positions the present work as a comparative study between an exact method and a heuristic approach. Chapter 3 presents the mathematical formulation of the considered problem. Chapter 4 describes the experimental setup, including the implementation of the model in Python and its solution with Gurobi for synthetic test instances. Chapter 5 develops the heuristic design. Chapter 6 evaluates the performance of the heuristic in comparison to the exact model. Chapter 7 concludes with a summary of findings and an outlook on future research.

2 The quay crane scheduling problem

2.1 Operational description and economic implications

Container terminals must coordinate vessel handling operations under strict time and capacity constraints. For terminal management, vessel turnaround time represents a key performance indicator, as it affects berth utilization, service reliability, and contractual compliance. Delays may result in financial penalties and reduced competitiveness.

When a vessel is berthed, a predefined workload of loading and unloading container moves has to be completed before departure. Several vessels may require service at the same time, which creates competition for limited quay crane capacity. The terminal operator must therefore decide how the available cranes are allocated among vessels and how the handling operations are scheduled over time.

Quay cranes are crucial operational resources. They move on the same rail track along the quay and are subject to technical and safety-related constraints. In particular, cranes cannot cross each other, and minimum safety distances must be maintained. Repositioning between working areas requires time, and each crane can only operate within the spatial boundaries of a vessel. These characteristics imply that crane assignments are interdependent and must be coordinated in an integrated manner. To illustrate the physical configuration of quay cranes relative to berthed vessels, Figure 1 provides a schematic overview of cranes servicing multiple vessel bays at a container terminal.

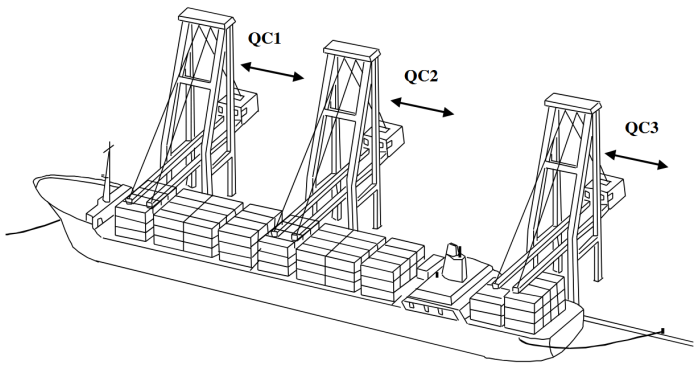

Figure 1: A figure of quay cranes working on a vessel.
Source: Kim and Park (2004)

Containers arriving at a terminal or to be loaded onto a vessel are typically grouped for operational planning. Inbound containers that originate from the same loading port, are of the same size, and are transported by the same vessel are considered to belong to the same container group; similarly, outbound containers that share the same destination port, container size, and designated vessel are said to be in the same container group.

In the literature, such container groups are commonly referred to as clusters (Kim and Park 2004).

A stowage plan records the planned positioning of clusters on a vessel prior to loading and after discharge. In practice, cluster positions on a vessel are identified by a three-dimensional coordinate system consisting of bays, rows, and tiers: bays represent cross-sections of the vessel from bow (front) to stern (back), rows denote positions across the width of each bay, and tiers indicate vertical stacking levels above and below deck (see Figure 2).

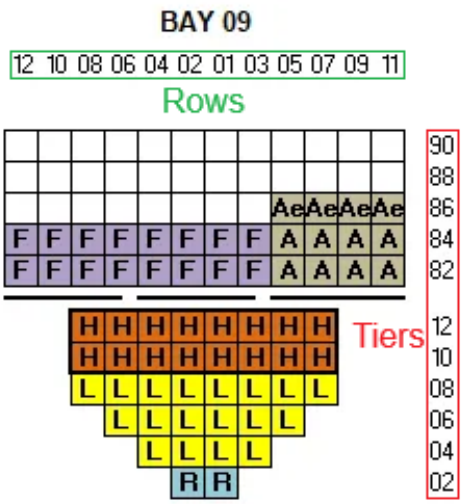

Figure 2: An example of a single bay stowage plan.
Source:
www.shippingandfreightresource.com/container-stowage-planning-and-how-it-works/

The spatial arrangement defined by bays, rows, and tiers has direct implications for quay crane operations: cranes must access specific positions on the vessel in an order that respects physical reachability and task precedence, since containers stacked above others or located deeper within a bay cannot be handled until obstructing containers are moved. Accordingly, the organization of container groups and clusters in the stowage plan influences both the sequence and timing of quay crane assignments and the overall schedule of loading and discharging operations.

From a managerial perspective, the QCSP consists of determining, for each vessel, the number and timing of assigned cranes as well as the sequence of handling operations, such that all container moves defined in the stowage plan are completed feasibly. These decisions must respect limited crane availability, non-crossing and safety constraints, vessel-specific workload distributions, and contractual departure requirements. Economically, the objective is to allocate crane capacity in a manner that minimizes vessel completion times and ensures schedule reliability, thereby increasing berth utilization

and terminal throughput while avoiding delay-related penalties, idle capacity, and additional operational costs.

2.2 Review of existing research

In the literature the QCSP is generally understood as a combinatorial decision problem concerned with assigning quay cranes to container handling tasks and determining the sequence in which these tasks are executed. While the operational setting is similar across studies – cranes operating on a common rail and serving a berthed vessel – the literature reveals substantial variation in how the problem is modeled and approached.

Early contributions established crane scheduling as an optimization problem with economic implications. Daganzo (1989) formulated crane allocation in ports as a cost-minimization problem, thereby framing it as a structured operations research problem. In a subsequent study, Peterkofsky and Daganzo (1990) modeled crane scheduling as a preemptive open-shop problem and proposed a branch-and-bound procedure. In this interpretation, tasks may be interrupted and resumed, and no fixed technological order between tasks is assumed (open-shop structure). These early models emphasize the combinatorial nature of the assignment and sequencing decisions but rely on simplified operational representations.

Later research introduced more detailed operational constraints that reflect the physical characteristics of quay cranes. A central modeling feature is the non-crossing constraint, which arises because cranes operate on the same rail and cannot pass each other. Zhu and Lim (2006) explicitly incorporate non-crossing restrictions. Their formulation assumes non-preemptive processing, meaning that once a crane starts a task, it must complete it without interruption. This distinction between preemptive and non-preemptive modeling is one of the structural differences across the literature.

Another key modeling dimension concerns the objective function. While early studies focused on delay-related cost measures (Daganzo 1989; Peterkofsky and Daganzo 1990), many later works adopt makespan minimization as the primary objective, interpreting vessel completion time as the dominant performance indicator. Kim and Park (2004) present a mixed-integer programming formulation for single-vessel quay crane scheduling that minimizes the vessel handling time under operational constraints. Their model assumes deterministic task durations and non-preemptive processing, and it has become a reference point for subsequent QCSP research.

Beyond objective choice and preemption assumptions, the literature differs substantially with respect to the scope of decision-making. Some studies focus exclusively on single-vessel handling (Kim and Park 2004; Zhu and Lim 2006), whereas others consider mul-

tiple vessels or integrate berth allocation and crane assignment decisions into a unified framework (Agra and Oliveira 2018). Furthermore, crane characteristics vary across models: cranes are often treated as homogeneous parallel machines, but some studies incorporate heterogeneous crane capabilities or additional yard-related constraints (Qin et al. 2020).

Methodologically, two principal paradigms can be identified. Exact approaches formulate the QCSP as a mixed-integer program or develop branch-and-bound or branch-and-cut algorithms (Kim and Park 2004; Moccia et al. 2006). These methods aim at proven optimality and are typically applied to relatively small or well-structured instances. In contrast, heuristic and metaheuristic approaches – such as tabu search (Sammarra et al. 2007), simulated annealing (Lashkari et al. 2017), or interference-aware constructive heuristics (Bierwirth and Meisel 2009) – prioritize computational tractability for larger instances and more complex constraint structures. More recent work combines both paradigms through hybrid procedures that embed exact components within metaheuristic frameworks (Zhao et al. 2020).

The review indicates that the QCSP does not represent a single, uniformly defined problem. Instead, it comprises a family of related models that differ along several dimensions:

- objective function,
- number of vessels and cranes considered,
- preemptive versus non-preemptive processing,
- open-shop versus precedence-constrained task structures,
- inclusion of interference and safety distance constraints,
- methodological orientation (exact optimization versus heuristic approaches).

Given these differences, results reported in literature are not directly comparable without careful consideration of modeling assumptions. For the purpose of the present thesis, a clearly delimited problem setting is required.

3 Mathematical Formulation

3.1 Modeling assumptions

In order to obtain a manageable representation of the QCSP, the real-world handling process is reduced to a simplified and well-defined analytical setting. The modeling approaches adopted in this thesis are based on the single-vessel formulation proposed by Kim and Park (2004), which has become a reference framework for deterministic quay crane scheduling under interference constraints.

The problem is restricted to the handling of a single berthed vessel. Interactions between multiple vessels and inter-ship crane reallocations are excluded. The vessel is assumed to have a fixed configuration consisting of 30 bays. The berth position and the stowage plan are treated as given inputs and are not subject to optimization.

Each handling task is assumed to have a fixed and known processing duration. Task execution is non-preemptive, meaning that once a crane has started a task, it must complete it without interruption. Furthermore, the model assumes a deterministic environment without operational disruptions, delays.

Precedence relations within each ship bay are explicitly considered. If both discharging and loading operations are required in the same bay, all discharging tasks must be completed before loading tasks can begin. For discharging operations, containers located on deck must be removed before containers in the hold of the same bay are handled. Conversely, for loading operations, containers in the hold must be placed before containers on deck. These precedence constraints reflect the physical accessibility conditions implied by the stowage structure.

Quay cranes are assumed to be homogeneous with respect to their technical capabilities and handling performance. They are initially spaced evenly along the vessel and operate on a single rail track parallel to the quay. Due to safety regulations and physical interference constraints, cranes cannot cross each other and must maintain minimum clearance distances. Consequently, certain tasks cannot be executed simultaneously if their execution would violate non-crossing or safety conditions.

The analysis focuses exclusively on quay crane scheduling decisions. Yard operations, hinterland transport processes, and other landside activities are excluded from consideration. Under these assumptions, the operational environment is reduced to a deterministic single-vessel scheduling problem with fixed processing times, defined precedence relations, and interference constraints. This formulation provides the foundation for the mathematical model developed in the subsequent section.

3.2 Notation

Based on the assumptions defined above, the deterministic single-vessel quay crane scheduling problem is formulated as a mixed-integer linear program (MILP). In the following, the notation is introduced in a structured and explanatory manner before presenting the objective function and constraints.

Handling operations derived from the stowage plan are represented as tasks. The set of all tasks is denoted by Ω. Individual tasks are indexed by $i, j \in \Omega$ and are ordered according to increasing bay position along the vessel. This spatial ordering is required for the formulation of non-crossing constraints. Quay cranes are indexed by $k = 1, \ldots, K$ and are likewise ordered according to their relative position along the quay.

Operational dependencies between tasks are captured through two subsets of ordered task pairs. The set $\Phi \subseteq \Omega \times \Omega$ contains all precedence relations. If $(i, j) \in \Phi$, task i must be completed before task j can start. These relations reflect discharge-before-load requirements as well as deck–hold ordering constraints within the same bay. In addition, the set $\Psi \subseteq \Omega \times \Omega$ contains all task pairs that are not allowed to be processed simultaneously. This set represents interference and safety distance restrictions implied by the shared crane rail.

All processing and travel times are assumed to be deterministic and known in advance. The parameter p_i denotes the processing time of task i, and l_i specifies its bay position. The earliest availability time of crane k is given by r_k, while l_k^0 represents its initial bay position. The travel time required for a crane to move from the bay of task i to the bay of task j is denoted by t_{ij}. Logical implications within the model are implemented using a sufficiently large positive constant M. The objective function is weighted by the non-negative parameters α_1 and α_2, where α_1 determines the relative importance of the makespan and α_2 weights the total crane completion time.

Routing and sequencing decisions are modeled by the binary variables X_{ij}^k. The variable X_{ij}^k equals 1 if crane k performs task j immediately after task i, and 0 otherwise. These successor variables determine the processing sequence of tasks on each crane. The continuous variable $D_i \geq 0$ denotes the completion time of task i, and $Y_k \geq 0$ represents the completion time of crane k. To model temporal ordering between potentially conflicting tasks, the binary variable Z_{ij} is introduced. It equals 1 if task j starts after task i has been completed, and 0 otherwise. Finally, the variable $W \geq 0$ denotes the vessel completion time (makespan), which is directly related to the vessel turnaround time.

For clarity and ease of reference, Table 1 provides a structured summary of the notation introduced in this section.

Table 1: Summary of notation

Category	Symbol	Description
Indices	i, j	Indices of handling tasks
	k	Index of quay cranes
Sets	Ω	Set of all handling tasks
	Φ	Set of precedence-related task pairs
	Ψ	Set of interference-related task pairs
Parameters	p_i	Processing time of task i
	l_i	Bay position of task i
	r_k	Earliest availability time of crane k
	l_k^0	Initial bay position of crane k
	t_{ij}	Travel time from task i to task j
	M	Sufficiently large constant
	α_1	Weight of the makespan in the objective function
	α_2	Weight of the total completion time in the objective function
Decision Variables	X_{ij}^k	1 if crane k executes j immediately after i, 0 otherwise
	D_i	Completion time of task i
	Y_k	Completion time of crane k
	Z_{ij}	1 if task j starts after completion of i, 0 otherwise
	W	Vessel completion time (makespan)

Having introduced the notation, the mathematical model is presented in the following subsection.

3.3 A mathematical formulation

After the modeling assumptions and notation have been introduced, the mathematical model can now be described. The formulation follows a routing-based mixed-integer programming approach. Each crane executes a sequence of tasks that starts at a dummy origin node and ends at a dummy terminal node. The model formulation presented in the following consists of the objective function to be minimized and the associated constraints.

$$\min \quad \alpha_1 W + \alpha_2 \sum_{k=1}^{K} Y_k \tag{1}$$

subject to

$$Y_k \leq W \qquad \forall k = 1, \ldots, K \tag{2}$$

$$\sum_{j \in \Omega} X_{0j}^k = 1 \qquad \forall k = 1, \ldots, K \tag{3}$$

$$\sum_{i \in \Omega} X_{iT}^k = 1 \qquad \forall k = 1, \ldots, K \tag{4}$$

$$\sum_{k=1}^{K} \sum_{i \in \Omega} X_{ij}^k = 1 \qquad \forall j \in \Omega \tag{5}$$

$$\sum_{j \in \Omega} X_{ij}^k - \sum_{j \in \Omega} X_{ji}^k = 0 \qquad \forall i \in \Omega,\ \forall k = 1, \ldots, K \tag{6}$$

$$D_i + t_{ij} + p_j - D_j \leq M(1 - X_{ij}^k) \qquad \forall i, j \in \Omega,\ \forall k = 1, \ldots, K \tag{7}$$

$$D_i + p_j \leq D_j \qquad \forall (i, j) \in \Phi \tag{8}$$

$$D_i - D_j + p_i \leq M(1 - Z_{ij}) \qquad \forall i, j \in \Omega \tag{9}$$

$$Z_{ij} + Z_{ji} = 1 \qquad \forall (i, j) \in \Psi \tag{10}$$

$$\sum_{v=1}^{k} \sum_{u \in \Omega} X_{uj}^v - \sum_{v=1}^{k} \sum_{u \in \Omega} X_{ui}^v \leq M(Z_{ij} + Z_{ji}) \qquad \forall i, j \in \Omega :\ l_i < l_j,\ \forall k = 1, \ldots, K \tag{11}$$

$$D_j + t_{jT}^k - Y_k \leq M(1 - X_{jT}^k) \qquad \forall j \in \Omega,\ \forall k = 1, \ldots, K \tag{12}$$

$$r_k - D_j + t_{0j}^k + p_j \leq M(1 - X_{0j}^k) \qquad \forall j \in \Omega,\ \forall k = 1, \ldots, K \tag{13}$$

$$X_{ij}^k,\ Z_{ij} \in \{0, 1\} \qquad \forall i, j \in \Omega,\ \forall k = 1, \ldots, K \tag{14}$$

$$Y_k,\ D_i \geq 0 \qquad \forall i \in \Omega,\ \forall k = 1, \ldots, K \tag{15}$$

3.3.1 Objective function

$$\min\ \alpha_1 W + \alpha_2 \sum_{k=1}^{K} Y_k \tag{1}$$

The objective function consists of two components. The primary objective is the minimization of the vessel completion time (makespan) W. Since a vessel can depart only after all assigned cranes have completed their tasks, the makespan directly determines

the vessel turnaround time. The second term minimizes the sum of crane completion times. This encourages earlier release of cranes and reduces overall crane occupation. By selecting $\alpha_1 \gg \alpha_2$, the makespan is minimized first, while the total crane completion time acts as a secondary performance criterion.

3.3.2 Makespan definition

The makespan variable is defined as an upper bound on the completion time of each crane:

$$Y_k \leq W \qquad \forall k = 1, \ldots, K \tag{2}$$

Together with the minimization of W, constraint (2) ensures that W equals the maximum crane completion time.

3.3.3 Assignment and routing constraints

Constraints (3)–(6) define the routing structure of the cranes and guarantee that each task is processed exactly once.

Each crane route starts at a dummy origin node 0 and selects exactly one first task:

$$\sum_{j \in \Omega} X_{0j}^k = 1 \qquad \forall k = 1, \ldots, K \tag{3}$$

Similarly, each crane must terminate its route at the dummy terminal node T:

$$\sum_{i \in \Omega} X_{iT}^k = 1 \qquad \forall k = 1, \ldots, K \tag{4}$$

Each task must be assigned to exactly one crane and must have exactly one predecessor:

$$\sum_{k=1}^{K} \sum_{i \in \Omega} X_{ij}^k = 1 \qquad \forall j \in \Omega \tag{5}$$

Flow conservation ensures that for every crane and every task, the number of incoming arcs equals the number of outgoing arcs:

$$\sum_{j\in\Omega} X_{ij}^k - \sum_{j\in\Omega} X_{ji}^k = 0 \qquad \forall i \in \Omega,\ \forall k = 1,\dots,K \tag{6}$$

To illustrate the routing logic, consider one crane and three tasks. Constraints (3) and (4) select exactly one start and one end task, while constraint (6) enforces balanced in- and out-degree for each visited task. The resulting structure is a single sequence of the form $0 \to i \to j \to h \to T$.

Constraints (3)–(6) therefore construct a single continuous route for each crane from node 0 to node T. Subtours that are disconnected from the dummy nodes are eliminated through the time-linking constraints introduced below.

3.3.4 Temporal feasibility constraints

If crane k performs task j immediately after task i, travel and processing times must be respected:

$$D_i + t_{ij} + p_j - D_j \le M\left(1 - X_{ij}^k\right) \qquad \forall i,j \in \Omega,\ \forall k = 1,\dots,K \tag{7}$$

If $X_{ij}^k = 1$, the inequality reduces to

$$D_j \ge D_i + t_{ij} + p_j,$$

which enforces chronological consistency along the crane route. If $X_{ij}^k = 0$, the constraint is relaxed by the Big-M term. Since completion times strictly increase along active arcs, disconnected cycles are infeasible, and subtours are thereby excluded.

3.3.5 Precedence constraints

Operational dependencies derived from the stowage plan are enforced by:

$$D_i + p_i \le D_j \qquad \forall (i,j) \in \Phi \tag{8}$$

For each ordered pair $(i,j) \in \Phi$, task i must be completed before task j can start. These constraints capture discharge-before-load relations and deck–hold sequencing requirements.

3.3.6 Interference and non-crossing constraints

Interference between tasks and the non-crossing property of quay cranes are modeled through additional binary ordering variables Z_{ij}.

Temporal ordering between two tasks is defined by:

$$D_i - D_j + p_i \leq M\,(1 - Z_{ij}) \qquad \forall i, j \in \Omega \tag{9}$$

If $Z_{ij} = 1$, the inequality implies

$$D_j \geq D_i + p_i,$$

which enforces that task i precedes task j in time. If $Z_{ij} = 0$, the constraint does not impose a temporal relation. Thus, Z_{ij} activates a precedence decision between the two tasks.

For task pairs that are not allowed to overlap due to safety or operational restrictions, exactly one of the two possible temporal orders must hold:

$$Z_{ij} + Z_{ji} = 1 \qquad \forall (i, j) \in \Psi \tag{10}$$

For each interfering pair $(i, j) \in \Psi$, either task i precedes task j or vice versa. Combined with constraint (9), this guarantees that tasks in Ψ cannot overlap in time.

The temporal ordering variables are further used to enforce the spatial non-crossing requirement of quay cranes:

$$\sum_{v=1}^{k} \sum_{u \in \Omega} X_{uj}^{v} - \sum_{v=1}^{k} \sum_{u \in \Omega} X_{ui}^{v} \leq M\,(Z_{ij} + Z_{ji}) \qquad \forall i, j \in \Omega:\ l_i < l_j,\ \forall k = 1, \ldots, K \tag{11}$$

Constraint (11) enforces the non-crossing of quay cranes operating on a common rail. Both tasks and cranes are ordered according to increasing bay position. The inner sum

$$\sum_{u \in \Omega} X_{uj}^{v}$$

equals 1 if task j is assigned to crane v, and 0 otherwise. Hence, the cumulative term

$$\sum_{v=1}^{k} \sum_{u \in \Omega} X_{uj}^{v}$$

equals 1 if task j is assigned to one of the cranes with index $1, \ldots, k$, and 0 otherwise. The same interpretation applies to task i.

Assume $l_i < l_j$, meaning that task i is located to the left of task j. If both tasks are processed simultaneously, then $Z_{ij} + Z_{ji} = 0$, and the right-hand side of (11) equals zero. The constraint therefore reduces to

$$\sum_{v=1}^{k} \sum_{u \in \Omega} X_{uj}^{v} - \sum_{v=1}^{k} \sum_{u \in \Omega} X_{ui}^{v} \leq 0 \qquad \forall k.$$

This inequality implies that whenever task j is assigned to one of the first k cranes, task i must also be assigned to one of the first k cranes. In other words, a task located further to the right cannot be assigned to a crane with a smaller index than a simultaneously processed task located to the left. Consequently, the spatial order of tasks ($l_i < l_j$) is consistent with the index order of cranes. Any assignment that would require two cranes to cross along the rail would violate (11).

Constraint (11) therefore couples spatial position, crane index, and temporal overlap into a single consistency condition and ensures that the relative order of cranes along the quay is preserved at all times.

3.3.7 Crane completion and availability constraints

The completion time of crane k is linked to its last executed task and the travel time to its terminal node:

$$D_j + t_{jT}^{k} - Y_k \leq M \left(1 - X_{jT}^{k}\right) \qquad \forall j \in \Omega,\ \forall k = 1, \ldots, K \tag{12}$$

Crane availability and initial positioning are enforced by:

$$r_k - D_j + t_{0j}^{k} + p_j \leq M \left(1 - X_{0j}^{k}\right) \qquad \forall j \in \Omega,\ \forall k = 1, \ldots, K \tag{13}$$

If $X_{0j}^{k} = 1$, task j cannot be completed before crane k becomes available and travels from its initial position to the bay of task j.

3.3.8 Domain restrictions

$$X_{ij}^{k},\ Z_{ij} \in \{0,1\} \qquad \forall i,j \in \Omega,\ \forall k = 1,\dots,K \tag{14}$$

$$Y_{k},\ D_{i} \geq 0 \qquad \forall i \in \Omega,\ \forall k = 1,\dots,K \tag{15}$$

Constraints (14) and (15) define the admissible domains of the decision variables. The routing and ordering variables are binary, while completion time variables are non-negative continuous variables.

4 Numerical experiments

The purpose of the experiments are to evaluate the computational performance and scalability of the proposed MILP formulation. In particular, the experiments are designed to analyze the growth of computational effort with increasing problem size, to determine the practical limits of exact solvability within a fixed time budget, and to investigate the influence of available computation time on solution quality.

4.1 Technical configuration

The MILP formulation is implemented in Python and solved using Gurobi 13.0.0. All experiments are conducted on a machine equipped with an AMD Ryzen 5 5600X processor and 32 GB DDR4 RAM (3200 MHz).

Unless stated otherwise, all solver parameters remain at their default settings. The solver is executed using a single computational thread in order to ensure consistent and reproducible performance measurements. The complete Python benchmark framework and detailed solver configuration are provided in Appendix A.

4.2 Experimental design

The numerical evaluation consists of three experiments.

The first experiment investigates scalability with respect to the number of handling tasks. The number of tasks is varied in increments of five, i.e., $|\Omega| \in \{5, 10, 15, 20\}$, while the number of quay cranes is set to $K \in \{2, 3\}$. For each $(|\Omega|, K)$ configuration, five independent instances are generated using different random seeds. In order to isolate the effect of increasing problem size, the number of bays is set equal to the number of tasks. This keeps the average task density per bay approximately constant

and ensures that observed performance changes are primarily due to the increase in decision variables and constraints.

The second experiment determines the practical solvability limit of the formulation under a fixed time limit of 1000 seconds. The physical layout is kept constant by fixing the number of bays to 30. Starting from small task sets, the number of tasks is increased stepwise. For each configuration, five independent instances are solved and the proportion of instances for which optimality can be certified is recorded. This experiment identifies the transition point at which the solver fails to prove optimality within the prescribed time limit.

The third experiment analyzes the influence of computation time on solution quality. Identical instances are solved repeatedly under different time limits. The final optimality gap is used as an indicator of solution quality. This experiment quantifies the trade-off between runtime and optimality certification.

4.3 Instance generation and parameterization

All instances are generated synthetically using a controlled pseudo-random procedure. Structural parameters such as the number of tasks, number of bays, and number of cranes are fixed according to the experimental design. Random seeds are stored to ensure reproducibility.

Each task i is assigned a bay position l_i drawn uniformly from $\{1, \ldots, \textit{num_bays}\}$. Processing times p_i are independently sampled from a continuous uniform distribution on the interval $[3, 180]$.

Precedence relations are generated locally within each bay. For each ordered pair of tasks located in the same bay, a precedence constraint is introduced with probability 0.3. No precedence relations are created across different bays.

The interference set Ψ is defined based on spatial adjacency. Two tasks interfere if and only if they are located in adjacent bays, i.e., $|l_i - l_j| = 1$. This models safety distance constraints between cranes operating in neighboring bays.

Travel times are proportional to the absolute difference in bay indices, with one time unit per bay difference. Quay cranes are initially positioned evenly along the vessel and return to their initial positions after completing their assigned tasks. All cranes are available at time zero.

The objective function consists of a weighted combination of performance components with parameters $\alpha_1 = 100$ and $\alpha_2 = 1$. The chosen weights prioritize the primary

scheduling objective while maintaining a secondary performance component with lower relative importance.

The Big-M parameter is computed individually for each instance as the sum of all processing times plus an upper bound on total crane travel time. Formally,

$$M = \sum_{i \in \Omega} p_i + (\textit{num_bays} - 1) \cdot |\Omega|.$$

The second term represents a worst-case estimate in which each task requires a crane to travel the maximum possible bay distance. Combined with the total processing time, this yields an upper bound on the makespan of any feasible schedule. This ensures that all conditional constraints remain valid, while keeping M sufficiently small to maintain numerical stability.

Table 2 summarizes the instance generation parameters and solver settings used throughout the numerical experiments.

Parameter	**Value/Range**
Number of cranes K	$\{2, 3\}$
Processing times p_i	$[3, 180]$
Precedence probability	0.3
Travel time	1 time unit per bay distance
Initial crane availability	0s
Objective weights	$\alpha_1 = 100$, $\alpha_2 = 1$
Solver threads	1

Table 2: Summary of instance parameters and experimental settings.

4.4 Results

4.4.1 Task scaling experiment

Table 3 summarizes the results of the task scaling experiment. The table reports the average optimality gap, the average runtime, as well as the number of instances solved to proven optimality and the number of time limit terminations.

		seeds	$\overline{gap}$ [%]	$\bar{t}$ [s]	*optimal*	*timelimit*
K = 2	$T = 5$	5	0.00	0.18	5	0
	$T = 10$	5	88.68	1000	0	5
	$T = 15$	5	94.83	1000	0	5
	$T = 20$	5	97.25	1000	0	5
K = 3	$T = 5$	5	0.00	0.23	5	0
	$T = 10$	5	70.72	1000	0	5
	$T = 15$	5	89.42	1000	0	5
	$T = 20$	5	95.16	1000	0	5

Table 3: Results of the task scaling experiment.

For $T = 5$, all instances are solved to proven optimality within negligible runtime for both $K = 2$ and $K = 3$. The average optimality gap is zero and no time-limit terminations occur.

A substantial decrease in performance is observed when increasing the number of tasks to $T = 10$. For both crane configurations, none of the five instances can be solved to proven optimality within the time limit of 1000 seconds. All runs terminate due to the imposed time limit, and the average optimality gap increases sharply to 88.68% for $K = 2$ and 70.72% for $K = 3$.

For $T \geq 15$, this pattern persists. All instances reach the time limit and the final optimality gaps remain at very high levels. The task scaling experiment thus reveals a distinct transition between $T = 5$ and $T = 10$, where Gurobi ceases to provide proven optimal solutions within the fixed computational budget.

4.4.2 Capacity test

The capacity test refines the analysis of the observed transition region between $T = 5$ and $T = 10$. To determine this boundary more precisely, the number of tasks is increased incrementally between these values under a fixed bay layout of 30. For each task size and crane configuration, five independent instances are generated and solved with a time limit of 1000 seconds. The proportion of instances solved to proven optimality is recorded.

This experiment identifies the smallest task size for which the solver fails to consistently

prove optimality within the prescribed time limit and therefore determines the practical solvability limit of the MILP formulation under the given computational budget.

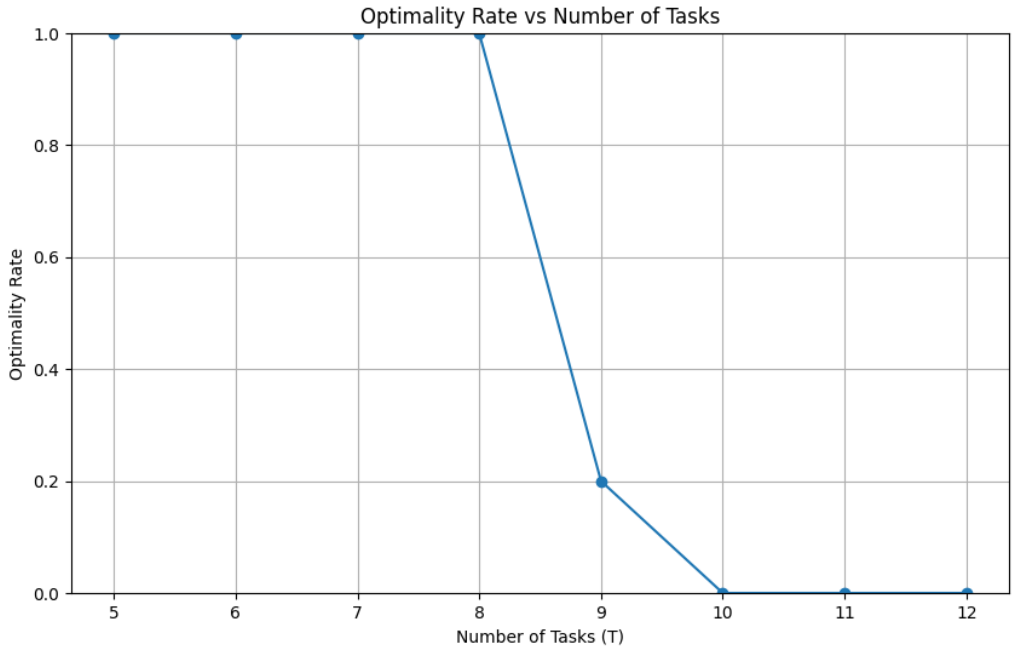

Figure 3: Optimality rate vs number of tasks.

Figure 3 shows that instances with up to $|\Omega| = 8$ tasks are solved to proven optimality within the time limit. For larger task counts, the proportion of instances solved to optimality decreases sharply, and most runs terminate due to the time limit.

4.4.3 Time limit experiment

The time limit experiment examines the influence of available computation time on solution quality and bound progression. A fixed configuration with $K = 3$, $|\Omega| = 10$, and *num_bays* $= 10$ is considered. For each of five random seeds, identical instances are solved under increasing time limits. The final optimality gap, the incumbent objective value (primal bound), and the best bound (dual bound) are recorded at termination.

Figure 4 shows that the optimality gap decreases as the time limit increases. The reduction is systematic across all instances. However, the improvement rate is moderate. Although the gap is reduced when extending the time limit from 1000 s to 2500 s, none of the instances reaches proven optimality within the considered time horizon. The decrease in gap becomes progressively smaller as runtime increases, suggesting diminishing marginal returns of additional computational effort.

To analyze the underlying mechanism, Figure 5 presents the evolution of the primal

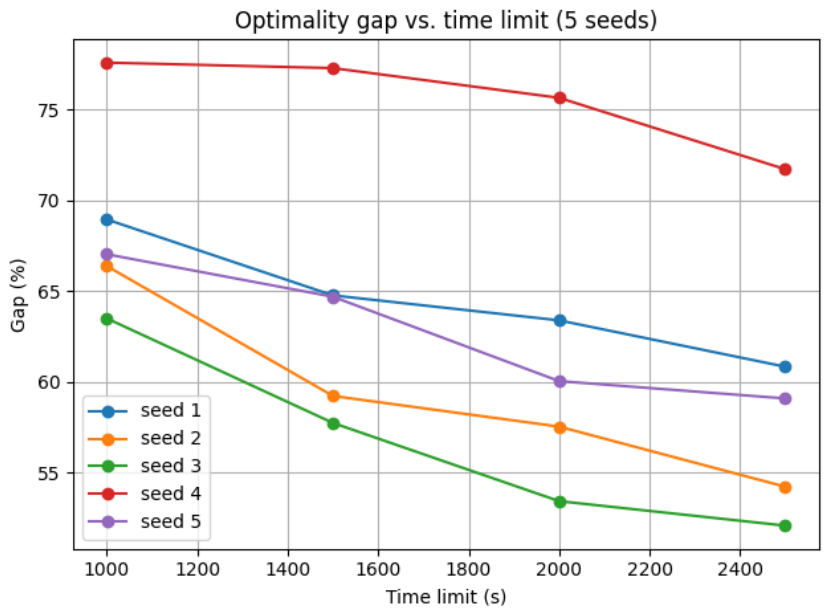

Figure 4: Optimality gap as a function of the time limit (five seeds).

objective value and the best bound over increasing time limits.

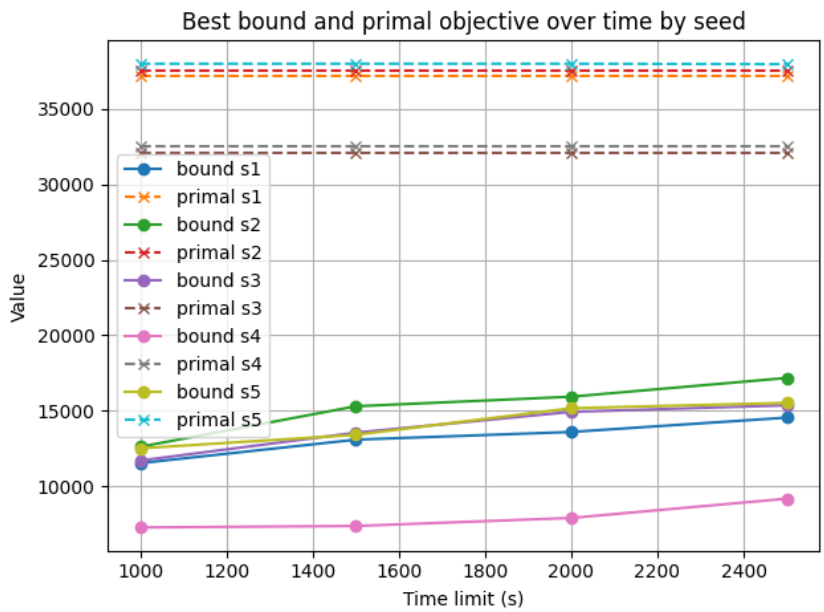

Figure 5: Evolution of primal objective and best bound over increasing time limits.

The figure indicates that the solver identifies a feasible incumbent solution at an early stage of the search process. The primal objective value stabilizes quickly. In contrast, the best bound progresses more slowly. Although the dual bound improves over time, the rate of improvement is limited and insufficient to close the remaining optimality gap within the tested time limits.

Taken together, the experimental results demonstrate that the MILP formulation pro-

vides reliable optimality certification only for small instances. While problems with up to eight tasks can be solved to proven optimality within the prescribed time limit, instances with nine or more tasks consistently fail to close the optimality gap. The time limit experiment further shows that feasible incumbent solutions are obtained rapidly, yet the dual bound improves only gradually. This behavior suggests that the computational bottleneck lies not in finding feasible schedules, but in tightening the dual bound sufficiently to close the gap. A potential explanation may be the presence of large Big-M coefficients and resulting weak linear programming relaxations, which can limit the strength of dual bounds and slow convergence.

Given that realistic terminal planning scenarios typically involve significantly more than nine tasks, exact solution approaches appear to be of limited practical applicability under strict time constraints. These observations motivate the investigation of heuristic methods, which aim to generate high-quality feasible solutions within short computation times without requiring optimality certification.

5 Heuristic Solution Approach

5.1 Design Principles

The heuristic follows a construction–improvement design. First, a feasible schedule is generated using a greedy rule. Second, the schedule is iteratively improved using tabu search.

A solution is represented as a mapping from quay cranes to ordered task sequences:

$$S = \{\pi_k \mid k \in \mathcal{K}\},$$

where each sequence π_k defines the processing order of tasks assigned to crane k. Each task appears exactly once across all sequences.

The timed schedule is obtained by computing start times sequentially. For each task, the start time is defined as the earliest time at which:

- the crane has completed its previous task and reached the required bay,
- all predecessor tasks have finished,
- no interference with adjacent cranes occurs.

If any of these conditions is not satisfied, the crane remains idle until the constraint is resolved. This waiting time ensures that all feasibility requirements are fulfilled.

The objective function is defined as

$$f(S) = \alpha_1 \cdot C_{\max} + \alpha_2 \cdot \sum_{i \in \Omega} C_i,$$

where C_i denotes the completion time of task i and C_{max} represents the makespan of the schedule.

The overall structure of the heuristic is summarized in Algorithm 1.

Algorithm 1: Greedy & Tabu search

```
InputInstance()
S* ← ∅
repeat
    Phase 1: S ← GreedyConstruction()
    Phase 2: S ← TabuSearchImprovement(S)
    if S* = ∅ or f(S) < f(S*) then
        S* ← S
    end if
until stopping criterion is satisfied
return S*
```

5.2 Construction Phase

The construction phase builds an initial feasible schedule by adding one task at a time to an incomplete solution. The current solution is represented by the crane sequences $\{\pi_k \mid k \in \mathcal{K}\}$, where each sequence contains the tasks already assigned to crane k in processing order. Initially, all tasks are unscheduled, i.e., $\Omega_{\text{uns}} = \Omega$. The procedure is repeated until all tasks have been assigned.

Step 1: Determine precedence-feasible tasks. Let Ω_{uns} denote the set of tasks that are unscheduled. From this set, the subset of precedence-feasible tasks is constructed as

$$\Omega_{\text{avail}} = \{i \in \Omega_{\text{uns}} \mid \text{all predecessors of } i \text{ are already scheduled}\}.$$

Only tasks in Ω_{avail} are considered in the current iteration. This ensures that precedence constraints are satisfied by construction.

Step 2: Generate candidate assignments. For each task $i \in \Omega_{\text{avail}}$ and each crane $k \in \mathcal{K}$, a candidate schedule is formed by appending i to the end of π_k. At this point, only the order decisions are specified; start times are not yet fixed.

Step 3: Evaluate candidates and compute completion times. Each candidate schedule is evaluated immediately. Start and completion times are recomputed sequentially along the crane sequences. A task can start only after:

- the crane has completed its previous task and reached the required bay (travel time),
- all predecessor tasks have finished (precedence),
- no simultaneous processing with conflicting tasks occurs (interference).

If any of these conditions is not satisfied, the task start is postponed and the crane remains idle until the earliest feasible start time is reached. The resulting completion time of task i under crane k is denoted by $C_i^{(k)}$.

Step 4: Select best candidate. Among all evaluated task–crane combinations, the pair ($i^\star$, $k^\star$) that yields the smallest completion time is selected. If multiple task–crane pairs result in identical completion times, the first encountered pair is selected.

Step 5: Update the partial schedule. Task $i^\star$ is permanently appended to the sequence of crane $k^\star$. The task is removed from Ω_{uns}, and the procedure returns to Step 1.

The construction phase terminates once Ω_{uns} is empty, resulting in a complete initial schedule that serves as the starting point for the improvement phase.

5.3 Improvement Phase

The greedy solution is refined using a tabu search procedure. Starting from the initial schedule, the algorithm iteratively explores neighboring solutions and replaces the current schedule by the best admissible neighbor.

Step 1: Neighborhood generation. In each iteration, a set of neighboring solutions is generated through systematic modification of the task sequences. The following local operators are applied:

- **Relocate**: remove a task from one crane sequence and insert it at a selected position in another crane sequence.
- **Swap**: exchange two tasks assigned to different cranes.
- **Intra-insert**: remove a task from a crane sequence and reinsert it at a different position within the same sequence.

The neighborhood is generated systematically by considering all admissible moves, with an optional limit on the number of neighbors.

Step 2: Evaluation. Each neighboring solution is evaluated using the same schedule generation procedure as in the construction phase. Start times are recomputed sequentially, ensuring feasibility with respect to precedence, interference, and travel constraints. The objective value is then calculated for comparison.

Step 3: Selection and tabu restriction. Among all evaluated neighbors, the best admissible solution is selected. A solution is considered admissible if the corresponding move is not currently stored in the tabu list or if it satisfies the aspiration criterion.

The tabu list stores attributes of recently executed moves. These move attributes remain prohibited for a fixed number of iterations to prevent immediate reversal and cycling.

Step 4: Aspiration criterion. A tabu move is accepted if it results in a solution that improves upon the globally best objective value found so far.

Step 5: Termination. The search terminates after a predefined number of iterations or when no improvement is observed for a specified number of consecutive iterations.

5.4 Illustrative Numerical Example

To illustrate the behavior of the proposed heuristic, a small numerical example is presented. The greedy phase constructs a feasible schedule by selecting the task–crane assignment with the smallest earliest completion time under the current partial solution. The tabu search phase can then improve the schedule by modifying local order decisions, while feasibility is preserved through the same sequential schedule evaluation procedure.

Input data

Two quay cranes are available, $\mathcal{K} = \{1, 2\}$, and five tasks must be scheduled, $\Omega = \{1, 2, 3, 4, 5\}$. Processing times p_i and bay locations l_i are given in Table 4.

Table 4: Input data for the numerical example

Task i	1	2	3	4	5
Bay l_i	2	3	5	6	9
Processing time p_i	6	5	4	7	3

The precedence relation $1 \to 2$ is assumed. Interference constraints prohibit simultaneous processing of tasks located in adjacent bays, i.e., tasks i and j cannot overlap in time if $|l_i - l_j| = 1$. Travel time equals one time unit per bay. Initial crane positions are set to $l_1^0 = 1$ and $l_2^0 = 10$, and both cranes are available at time 0.

The objective is

$$f(S) = 100 \cdot C_{\max} + \sum_{i \in \Omega} C_i,$$

where α_1 = 100 and α_2 = 1.

Phase 1: Greedy construction

Initially, both sequences are empty: $\pi_1 = ()$, $\pi_2 = ()$.

Iteration 1. Since Task 2 requires Task 1, the available set is $\Omega_{\text{avail}} = \{1, 3, 4, 5\}$. For each (i, k), the completion time is computed from travel time and processing time (no interference occurs yet). The smallest completion time is obtained by assigning Task 5 to Crane 2:

$$l_2^0 = 10 \to l_5 = 9: \text{ travel } 1,\ S_5 = 1,\ C_5 = 4.$$

Update: $\pi_2 = (5)$, $\pi_1 = ()$.

Iteration 2. The available set is $\Omega_{\text{avail}} = \{1, 3, 4\}$. The smallest completion time is obtained by assigning Task 1 to Crane 1:

$$l_1^0 = 1 \to l_1 = 2: \text{ travel } 1,\ S_1 = 1,\ C_1 = 7.$$

Update: $\pi_1 = (1)$, $\pi_2 = (5)$.

Iteration 3. Task 2 becomes available, hence $\Omega_{\text{avail}} = \{2, 3, 4\}$. Assigning Task 2 to Crane 2 would yield an early arrival, but it cannot start before Task 1 is completed and it cannot overlap with Task 1 due to adjacent bays ($|2 - 3| = 1$). The best candidate is Task 3 on Crane 2:

$$\text{Crane 2 after Task 5: } l_5 = 9 \to l_3 = 5: \text{ travel } 4,\ S_3 = 8,\ C_3 = 12.$$

Update: $\pi_2 = (5, 3)$, $\pi_1 = (1)$.

Iteration 4. The available set is $\Omega_{\text{avail}} = \{2, 4\}$. Task 4 on Crane 1 would require waiting because Task 3 on Crane 2 is processed in bay 5 from 8 to 12, and $|5-6| = 1$. Thus, Crane 1 would have to delay Task 4 in bay 6 until time 12. The best candidate is

Task 2 on Crane 1:

$$\text{Crane 1 after Task 1: } l_1 = 2 \to l_2 = 3: \text{ travel } 1,\ S_2 = 8,\ C_2 = 13.$$

Update: $\pi_1 = (1, 2)$, $\pi_2 = (5, 3)$.

Iteration 5. Only Task 4 remains. Assigning Task 4 to Crane 2 yields:

$$\text{Crane 2 after Task 3: } l_3 = 5 \to l_4 = 6: \text{ travel } 1,\ S_4 = 13,\ C_4 = 20.$$

Final greedy solution:

$$\pi_1 = (1, 2), \qquad \pi_2 = (5, 3, 4).$$

The resulting timed schedule is summarized in Table 5. The makespan equals $C_{\max} = 20$ and $\sum_i C_i = 7 + 13 + 12 + 20 + 4 = 56$, hence $f(S) = 100 \cdot 20 + 56 = 2056$.

Table 5: Timed schedule obtained by greedy construction

Task i	Crane k	Bay l_i	Start S_i	Completion C_i
1	1	2	1	7
2	1	3	8	13
5	2	9	1	4
3	2	5	8	12
4	2	6	13	20

Phase 2: Tabu search improvement (one improving move)

Starting from the greedy solution, tabu search evaluates neighboring schedules created by relocate, swap, and intra-insert moves. Consider the following intra-insert move on Crane 2, which changes the order of Tasks 3 and 4:

$$\pi_2 = (5, 3, 4) \ \to\ \pi_2' = (5, 4, 3).$$

Re-evaluating the schedule yields the following times for Crane 2:

$$S_5 = 1,\ C_5 = 4, \qquad l_5 = 9 \to l_4 = 6: \text{ travel } 3,\ S_4 = 7,\ C_4 = 14,$$

$$l_4 = 6 \to l_3 = 5: \text{ travel } 1,\ S_3 = 15,\ C_3 = 19.$$

Crane 1 remains unchanged with $(1, 2)$ and completion time $C_2 = 13$. Since Crane 2 now finishes at 19, the makespan improves to $C_{\max} = 19$. The total completion time becomes $7 + 13 + 19 + 14 + 4 = 57$, resulting in an improved objective value of

$$f(S') = 100 \cdot 19 + 57 = 1957.$$

Table 6 summarizes the improved timed schedule.

Table 6: Timed schedule after an improving intra-insert move

Task i	Crane k	Bay l_i	Start S_i	Completion C_i
1	1	2	1	7
2	1	3	8	13
5	2	9	1	4
4	2	6	7	14
3	2	5	15	19

6 Computational Experiments

This section evaluates the performance of the proposed greedy–tabu heuristic and compares it with the MILP formulation introduced in Section 3. The instance generation procedure and technical configuration are identical to those used in the MILP experiments. This ensures that both approaches are evaluated under consistent conditions and that differences in performance can be attributed solely to the solution method.

6.1 Benchmark Design

The benchmark is divided into two parts. The first part considers small problem instances for which the MILP formulation can reliably prove optimality within the imposed time limit. These instances provide a reference for evaluating the solution quality of the heuristic.

The second part focuses on larger instances where the MILP solver frequently fails to prove optimality within the time limit. In this setting, the comparison concentrates on the ability of each method to identify high-quality feasible solutions within limited computation time.

Part A: Optimality comparison for small instances. The first experiment examines instances with a limited number of tasks, specifically $|\Omega| \leq 8$. For these instance sizes, the MILP solver is able to prove optimality within the predefined time limit. The optimal objective value obtained from the MILP formulation therefore serves as a reference solution.

For each instance, the heuristic solution is compared to the optimal MILP objective value. The comparison focuses on whether the heuristic reproduces the optimal solution and on the magnitude of the deviation when optimality is not reached.

Part B: Performance comparison for larger instances. The second experiment investigates instances that exceed the practical solvability limit of the MILP formulation. In this range, the MILP solver typically identifies feasible solutions but fails to close the optimality gap within the imposed time limit.

In this setting, the comparison focuses on the quality of feasible solutions produced within limited runtime. Specifically, the objective value of the heuristic solution is compared with the best feasible solution (primal bound) obtained by the MILP solver. Additionally, the development of the best feasible objective value over time is analyzed to determine which method is able to identify high-quality solutions earlier during the search process.

6.2 Heuristic Parameter Settings

The greedy construction phase does not require additional parameters, as tasks are assigned deterministically according to the earliest feasible completion time rule described in Section 5. In contrast, the tabu search improvement phase relies on several parameters that control the search behavior.

The most important parameter is the *tabu tenure*, which determines how long a move remains stored in the tabu list after it has been applied. During this period, the reverse move is temporarily forbidden in order to prevent cycling and repeated exploration of recently visited solutions.

To further reduce cycling effects, a small random variation is added to the tabu tenure. This mechanism, referred to as *tabu jitter*, slightly varies the duration for which a move remains tabu. As a result, the search trajectory becomes less predictable and the risk of returning to previously visited regions of the solution space is reduced.

The search process is also controlled by two stopping parameters. First, a maximum number of iterations limits the total computational effort of the tabu search. Second,

the algorithm terminates early if no improvement of the best solution has been observed for a predefined number of consecutive iterations.

The parameter values used in the computational experiments are summarized in Table 7. These values were selected based on preliminary tests and remained fixed for all benchmark instances.

Table 7: Parameter settings of the tabu search heuristic

Parameter	Value
Tabu tenure	max(3, 10% of $\|\Omega\|$)
Tabu jitter	3
Maximum iterations	100000
No-improvement limit	predefined limit

References

Agra, Agostinho and Maryse Oliveira (2018). "MIP approaches for the integrated berth allocation and quay crane assignment and scheduling problem." In: *European Journal of Operational Research* 264.1, pp. 138–148. DOI: 10.1016/j.ejor.2017.05.040.

Bierwirth, Christian and Frank Meisel (2009). "A fast heuristic for quay crane scheduling with interference constraints." In: *Journal of Scheduling* 12.4, pp. 345–360. DOI: 10.1007/s10951-009-0105-0.

Daganzo, Carlos F. (1989). "The crane scheduling problem." In: *Transportation Research Part B: Methodological* 23.3, pp. 159–175. DOI: 10.1016/0191-2615(89)90001-5.

Kim, Kap Hwan and Young-Man Park (Aug. 1, 2004). "A crane scheduling method for port container terminals." In: *European Journal of Operational Research* 156.3, pp. 752–768. DOI: 10.1016/S0377-2217(03)00133-4.

Lashkari, Shabnam, Yong Wu, and Matthew E. H. Petering (2017). "Sequencing dual-spreader crane operations: Mathematical formulation and heuristic algorithm." In: *European Journal of Operational Research* 262.2, pp. 521–534. DOI: 10.1016/j.ejor.2017.03.046.

Moccia, Luigi, Jean-François Cordeau, Manlio Gaudioso, and Gilbert Laporte (2006). "A branch-and-cut algorithm for the quay crane scheduling problem in a container terminal." In: *Naval Research Logistics* 53.1, pp. 45–59. DOI: 10.1002/nav.20121.

Peterkofsky, Roy I. and Carlos F. Daganzo (1990). "A branch and bound solution method for the crane scheduling problem." In: *Transportation Research Part B: Methodological* 24.3, pp. 159–172. DOI: 10.1016/0191-2615(90)90014-P.

Qin, Tianbao, Yuquan Du, Jiang Hang Chen, and Mei Sha (2020). "Combining mixed integer programming and constraint programming to solve the integrated scheduling problem of container handling operations of a single vessel." In: *European Journal of Operational Research* 285.3, pp. 884–901. DOI: 10.1016/j.ejor.2020.02.021.

Sammarra, Marcella, Jean-François Cordeau, Gilbert Laporte, and Maria Monaco (2007). "A tabu search heuristic for the quay crane scheduling problem." In: *Journal of Scheduling* 10.4-5, pp. 327–336. DOI: 10.1007/s10951-007-0029-5.

Zhao, Guodong, Jiyin Liu, Lixin Tang, Rong Zhao, and Yong Dong (2020). "Model and heuristic solutions for the multiple double-load crane scheduling problem in slab yards." In: *IEEE Transactions on Automation Science and Engineering* 17.3, pp. 1307–1319. DOI: 10.1109/TASE.2019.2946196.

Zhu, Yi and Andrew Lim (2006). "Crane scheduling with non-crossing constraint." In: *Journal of the Operational Research Society* 57.12, pp. 1464–1471. DOI: 10.1057/palgrave.jors.2602110.

A Python code

Source code 1: MILP model

```python
import gurobipy as gp
from gurobipy import GRB
from qc_problem import QCProblem

def build_model(problem):
    model = gp.Model("QC␣Schedule␣Model")

    X = {}
    for k in problem.qcs:
        for j in problem.tasks:
            X[(k,0,j)] = model.addVar(vtype=GRB.BINARY, name = f"X_{k}_0_{j}")
        for j in problem.tasks:
            X[(k,j,'T')] = model.addVar(vtype=GRB.BINARY, name = f"X_{k}_{j}_T")
        for i in problem.tasks:
            for j in problem.tasks:
                if i != j:
                    X[(k,i,j)] = model.addVar(vtype=GRB.BINARY, name = f"X_{k}_{i}_{j}")

    Y = {}
    for k in problem.qcs:
        Y[k] = model.addVar(vtype=GRB.CONTINUOUS, name = f"Y_{k}")

    D = {}
    for i in problem.tasks:
        D[i] = model.addVar(vtype=GRB.CONTINUOUS, name = f"D_{i}")

    Z = {}
    for i in problem.tasks:
        for j in problem.tasks:
            if i != j:
                Z[(i,j)] = model.addVar(vtype=GRB.BINARY, name = f"Z_{i}_{j}")

    W = model.addVar(vtype=GRB.CONTINUOUS, name = f"W")

    return model, X, Y, D, Z, W

def add_constraints(model, problem, X, Y, D, Z, W):

    for k in problem.qcs:
        model.addConstr(
            Y[k] <= W, name=f"makespan_{k}"
            )

    for k in problem.qcs:
        model.addConstr(
            gp.quicksum(X[(k, 0, j)] for j in problem.tasks) == 1, name=f"start_{k}"
            )

    for k in problem.qcs:
        model.addConstr(
            gp.quicksum(X[(k, i, 'T')] for i in problem.tasks) == 1, name=f"end_{k}"
            )

    for j in problem.tasks:
```

```
        sum_expr = gp.LinExpr()
        for k in problem.qcs:
            for i in [0] + problem.tasks:
                if i == j:
                    continue
                if (k,i,j) in X:
                    sum_expr += X[(k,i,j)]
        model.addConstr(sum_expr == 1, name=f"task_assigned_{j}")

    for k in problem.qcs:
        for i in problem.tasks:
            inflow = gp.LinExpr()
            outflow = gp.LinExpr()

            for h in [0] + problem.tasks:
                if h == i:
                    continue
                if (k,h,i) in X:
                    inflow += X[(k,h,i)]

            for j in problem.tasks + ['T']:
                if j == i:
                    continue
                if (k,i,j) in X:
                    outflow += X[(k,i,j)]

            model.addConstr(inflow == outflow, name = f"flow_{k}_{i}")

    for k in problem.qcs:
        for i in [0] + problem.tasks:
            for j in problem.tasks:
                if i == j or (k,i,j) not in X:
                    continue
                if i == 0:
                    model.addConstr(
                        problem.earliest_time[k] + problem.starting_travel_time.get((k,j),0)
                        + problem.duration[j] - D[j] <= problem.M * (1 - X[(k,0,j)]), name=f"comp_time_start_{k}_{j}"
                    )
                else:
                    model.addConstr(
                        D[i] + problem.travel_time.get((k,i, j), 0) + problem.duration[j] - D[j]
                        <= problem.M * (1 - X[(k, i, j)]),
                        name=f"comp_time_{k}_{i}_{j}"
                    )

    for (i,j) in problem.phi:
            model.addConstr(
                D[i] + problem.duration[j] <= D[j], name = f"task_order_{i}_{j}"
            )

    for i in problem.tasks:
        for j in problem.tasks:
            if i != j:
                model.addConstr(
                    D[i] - D[j] + problem.duration[j] <= problem.M * (1 - Z[(i,j)]),
                    name=f"Z_{i}_{j}"
                )
```

```
    for (i,j) in problem.psi:
        model.addConstr(
            Z[(i,j)] + Z[(j,i)] == 1, name= f"Z_constraint_{i}_{j}"
            )

    sorted_qcs = sorted(problem.qcs)

    for i in problem.tasks:
        for j in problem.tasks:
            if i != j and problem.location[i] < problem.location[j]:
                for k_val in range(1, len(sorted_qcs) + 1):
                    first_k_qcs = sorted_qcs[:k_val]
                    sum_j = gp.LinExpr()
                    sum_i = gp.LinExpr()

                    for qc in first_k_qcs:
                        for prev in [0] + problem.tasks:
                            if prev != j and (qc, prev, j) in X:
                                sum_j += X[(qc, prev, j)]

                        for prev in [0] + problem.tasks:
                            if prev != i and (qc, prev, i) in X:
                                sum_i += X[(qc, prev, i)]

                    model.addConstr(
                        sum_j - sum_i <= problem.M * (Z[(i, j)] + Z[(j, i)]),
                        name=f"interference_{i}_{j}_{k_val}"
                    )

    for k in problem.qcs:
        for j in problem.tasks:
            if (k,j,'T') in X:
                model.addConstr(
                    D[j] + problem.final_travel_time.get((k,j),0) - Y[k] <= problem.M * (1 - X[(k,j,'T')]), name=f"completion_time_{k}_{j}"
                )

    for k in problem.qcs:
        for j in problem.tasks:
            if (k,0,j) in X:
                model.addConstr(
                    problem.earliest_time[k] - D[j] + problem.starting_travel_time.get((k,j),0) + problem.duration[j] <= problem.M * (1 - X[(k,0,j)]),
                    name=f"earliest_start_{k}_{j}"
                )

def set_objective(model, problem, Y, W):
    model.setObjective(problem.alpha1 * W + problem.alpha2 * gp.quicksum(Y[k] for k in problem.qcs), GRB.MINIMIZE)
```

Source code 2: MILP solver

```python
import gurobipy as gp
from gurobipy import GRB

def solve_milp(
    problem,
    time_limit: float = None,
    mip_gap: float = None,
    verbose: bool = False,
    log_file: str = None
):
    from milp_model import build_model, add_constraints, set_objective

    model, X, Y, D, Z, W = build_model(problem)
    add_constraints(model, problem, X, Y, D, Z, W)
    set_objective(model, problem, Y, W)

    if time_limit is not None:
        model.setParam(GRB.Param.TimeLimit, time_limit)

    if mip_gap is not None:
        model.setParam(GRB.Param.MIPGap, mip_gap)

    model.setParam(GRB.Param.LogToConsole, 1 if verbose else 0)
    model.setParam(GRB.Param.Threads, 1)
    if log_file is not None:
        model.setParam(GRB.Param.LogFile, log_file)

    model.optimize()

    status_map = {
        GRB.OPTIMAL: "OPTIMAL",
        GRB.TIME_LIMIT: "TIME_LIMIT",
        GRB.INFEASIBLE: "INFEASIBLE",
        GRB.UNBOUNDED: "UNBOUNDED",
        GRB.INF_OR_UNBD: "INF_OR_UNBD"
    }

    result = {
        "status": status_map.get(model.Status, f"STATUS_{model.Status}"),
        "runtime": model.Runtime,
        "objective": model.ObjVal if model.SolCount > 0 else None,
        "makespan": W.X if model.SolCount > 0 else None,
        "task_completion": {},
        "qc_completion": {},
        "routes": {},
        "best_bound": model.ObjBound,
        "gap": model.MIPGap if model.SolCount > 0 else None,
        "node_count": model.NodeCount,
        "sol_count": model.SolCount
    }

    def _extract_routes(problem, X):
        routes = {}

        for k in problem.qcs:
            route = []
            current = 0
            visited = set()

            while True:
                next_task = None
```

```
                for j in problem.tasks + ['T']:
                    if (k, current, j) in X and X[(k, current, j)].X > 0.5:
                        next_task = j
                        break

                if next_task is None or next_task == 'T':
                    break

                if next_task in visited:
                    print(f"WARNING: cycle detected in QC {k}, stopping route extraction")
                    break

                visited.add(next_task)
                route.append(next_task)
                current = next_task

            routes[k] = route

        return routes

    if model.SolCount > 0:

        for i in problem.tasks:
            result["task_completion"][i] = D[i].X

        for k in problem.qcs:
            result["qc_completion"][k] = Y[k].X

        result["routes"] = _extract_routes(problem, X)

    return result
```

Source code 3: Instance generator and problem data

```python
import random
from typing import Dict, List, Tuple
from dataclasses import dataclass

@dataclass(frozen=True)
class Task:
    id: int
    bay: int
    duration: float

class QCProblem:
    def __init__(self):
        pass

    @classmethod
    def generate(
        cls,
        num_bays: int,
        num_qcs: int,
        total_tasks: int,
        processing_time_range: Tuple[float, float] = (3.0, 180.0),
        travel_per_bay: float = 1.0,
        precedence_probability: float = 0.3,
        seed: int = None
    ):
        if seed is not None:
            random.seed(seed)

        prob = cls()
        prob.qcs = list(range(1, num_qcs + 1))
        tasks: List[Task] = []

        for tid in range(1, total_tasks + 1):
            bay = random.randint(1, num_bays)
            duration = random.uniform(*processing_time_range)

            tasks.append(
                Task(
                    id=tid,
                    bay=bay,
                    duration=duration
                )
            )

        tasks.sort(key=lambda t: t.bay)
        id_map = {old.id: i + 1 for i, old in enumerate(tasks)}

        tasks = [
            Task(
                id=id_map[t.id],
                bay=t.bay,
                duration=t.duration
            )
            for t in tasks
        ]

        prob.tasks = [t.id for t in tasks]
        prob.duration = {t.id: t.duration for t in tasks}
        prob.location = {t.id: t.bay for t in tasks}
        prob.phi = []
        by_bay: Dict[int, List[Task]] = {}

        for t in tasks:
```

```
            by_bay.setdefault(t.bay, []).append(t)

        for bay, bay_tasks in by_bay.items():
            for i in range(len(bay_tasks)):
                for j in range(i + 1, len(bay_tasks)):
                    if random.random() < precedence_probability:
                        prob.phi.append((bay_tasks[i].id, bay_tasks[j].id))

        prob.psi = []
        for i in range(len(tasks)):
            for j in range(i + 1, len(tasks)):
                if abs(tasks[i].bay - tasks[j].bay) == 1:
                    prob.psi.append((tasks[i].id, tasks[j].id))

        prob.earliest_time = {k: 0.0 for k in prob.qcs}
        prob.initial_position = {}
        for k in prob.qcs:
            if num_qcs > 1:
                position = int((k - 1) * (num_bays - 1) / (num_qcs - 1)) + 1
            else:
                position = 1
            prob.initial_position[k] = position

        prob.final_position = {k: prob.initial_position[k] for k in prob.qcs}

        prob.starting_travel_time = {}
        prob.travel_time = {}
        prob.final_travel_time = {}

        for k in prob.qcs:
            start_bay = prob.initial_position[k]
            end_bay = prob.final_position[k]

            for t in tasks:
                prob.starting_travel_time[(k, t.id)] = travel_per_bay * abs(start_bay - t.bay)

            for ti in tasks:
                for tj in tasks:
                    prob.travel_time[(k, ti.id, tj.id)] = travel_per_bay * abs(ti.bay - tj.bay)

            for t in tasks:
                prob.final_travel_time[(k, t.id)] = travel_per_bay * abs(t.bay - end_bay)

        total_processing_time = sum(prob.duration.values())
        max_travel_time = travel_per_bay * (num_bays - 1) * len(tasks)
        prob.M = total_processing_time + max_travel_time

        prob.alpha1 = 100
        prob.alpha2 = 1

        return prob
```

Source code 4: MILP benchmark

```
from __future__ import annotations
```

```python
import csv
from dataclasses import dataclass, asdict
from typing import Dict, Iterable, List, Optional

from qc_problem import QCProblem
from milp_solver import solve_milp

@dataclass
class ExperimentConfig:
    experiment_type: str
    K: Optional[int] = None
    T: Optional[int] = None
    num_bays: Optional[int] = None
    time_limit: Optional[float] = None
    mip_gap: Optional[float] = None
    seeds: Optional[List[int]] = None
    time_limits: Optional[List[float]] = None

def run_single_instance(
    problem: QCProblem,
    config: ExperimentConfig,
    seed: int,
    time_limit: float,
    mip_gap: Optional[float],
) -> Dict[str, object]:
    result = solve_milp(problem, time_limit=time_limit, mip_gap=
       mip_gap, verbose=False)
    gap = result.get("gap")
    gap_percent = gap * 100 if gap is not None else None

    return {
        "experiment_type": config.experiment_type,
        "K": config.K,
        "T": config.T,
        "num_bays": config.num_bays,
        "seed": seed,
        "time_limit": time_limit,
        "status": result.get("status"),
        "objective": result.get("objective"),
        "makespan": result.get("makespan"),
        "gap_percent": gap_percent,
        "runtime": result.get("runtime"),
        "best_bound": result.get("best_bound"),
        "node_count": result.get("node_count"),
        "sol_count": result.get("sol_count"),
    }

def write_results_csv(
    file_path: str,
    rows: Iterable[Dict[str, object]],
    fieldnames: List[str],
    metadata: Optional[Dict[str, object]] = None,
) -> None:
    with open(file_path, "w", newline="") as f:
        if metadata:
            for k, v in metadata.items():
                f.write(f"# {k}: {v}\n")
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for row in rows:
            writer.writerow(row)

def run_task_scaling(
    seeds: List[int],
```

```
    time_limit: float,
    output_csv: str,
) -> None:
    rows: List[Dict[str, object]] = []

    Ks = [2, 3]
    Ts = [5, 10, 15, 20]

    for K in Ks:
        for T in Ts:
            num_bays = T
            config = ExperimentConfig(
                experiment_type="task_scaling",
                K=K,
                T=T,
                num_bays=num_bays,
                time_limit=time_limit,
                mip_gap=None,
                seeds=seeds,
            )
            for seed in seeds:
                problem = QCProblem.generate(
                    num_bays=num_bays,
                    num_qcs=K,
                    total_tasks=T,
                    seed=seed,
                )
                row = run_single_instance(problem, config, seed,
                   time_limit, None)
                rows.append(row)

    metadata = {
        "experiment_type": "task_scaling",
        "Ks": Ks,
        "Ts": Ts,
        "num_bays_rule": "equal_to_T",
        "time_limit": time_limit,
        "mip_gap": None,
        "seeds": seeds,
    }
    fieldnames = [
        "experiment_type",
        "K",
        "T",
        "num_bays",
        "seed",
        "time_limit",
        "status",
        "objective",
        "makespan",
        "gap_percent",
        "runtime",
        "best_bound",
        "node_count",
        "sol_count",
    ]
    write_results_csv(output_csv, rows, fieldnames, metadata)

def run_capacity_test(
    Ks: List[int],
    Ts: List[int],
    seeds: List[int],
    time_limit: float,
    output_csv: str,
```

```
) -> None:
    rows: List[Dict[str, object]] = []
    num_bays = 30

    for K in Ks:
        for T in Ts:
            config = ExperimentConfig(
                experiment_type="capacity_test",
                K=K,
                T=T,
                num_bays=num_bays,
                time_limit=time_limit,
                mip_gap=None,
                seeds=seeds,
            )
            for seed in seeds:
                problem = QCProblem.generate(
                    num_bays=num_bays,
                    num_qcs=K,
                    total_tasks=T,
                    seed=seed,
                )
                row = run_single_instance(problem, config, seed,
                   time_limit, None)
                rows.append(row)

    metadata = {
        "experiment_type": "capacity_test",
        "Ks": Ks,
        "Ts": Ts,
        "num_bays": num_bays,
        "time_limit": time_limit,
        "mip_gap": None,
        "seeds": seeds,
    }
    fieldnames = [
        "experiment_type",
        "K",
        "T",
        "num_bays",
        "seed",
        "time_limit",
        "status",
        "objective",
        "makespan",
        "gap_percent",
        "runtime",
        "best_bound",
        "node_count",
        "sol_count",
    ]
    write_results_csv(output_csv, rows, fieldnames, metadata)

def run_time_limit(
    K: int,
    T: int,
    num_bays: int,
    seeds: List[int],
    time_limits: List[float],
    output_csv: str,
) -> None:
    rows: List[Dict[str, object]] = []

    problems: Dict[int, QCProblem] = {
```

```
        seed: QCProblem.generate(
            num_bays=num_bays,
            num_qcs=K,
            total_tasks=T,
            seed=seed,
        )
        for seed in seeds
    }

    for seed, problem in problems.items():
        config = ExperimentConfig(
            experiment_type="time_limit",
            K=K,
            T=T,
            num_bays=num_bays,
            time_limit=None,
            mip_gap=None,
            seeds=seeds,
            time_limits=time_limits,
        )
        for tl in time_limits:
            row = run_single_instance(problem, config, seed, tl, None
                )
            rows.append(row)

    metadata = {
        "experiment_type": "time_limit",
        "K": K,
        "T": T,
        "num_bays": num_bays,
        "time_limits": time_limits,
        "mip_gap": None,
        "seeds": seeds,
    }
    fieldnames = [
        "experiment_type",
        "K",
        "T",
        "num_bays",
        "seed",
        "time_limit",
        "status",
        "objective",
        "makespan",
        "gap_percent",
        "runtime",
        "best_bound",
        "node_count",
        "sol_count",
    ]
    write_results_csv(output_csv, rows, fieldnames, metadata)

def main() -> None:
    fixed_seeds = list(range(1, 6))

    #1. task scaling
    run_task_scaling(
        seeds=fixed_seeds,
        time_limit=1000.0,
        output_csv="task_scaling.csv",
    )
    #2. capacity test
    run_capacity_test(
        Ks=[2, 3],
```

```
        Ts=[5, 6, 7, 8, 9, 10, 11, 12],
        seeds=fixed_seeds,
        time_limit=600.0,
        output_csv="capacity_test.csv",
    )

    # 3. time-limit/anytime curve
    run_time_limit(
        K=3,
        T=10,
        num_bays=10,
        seeds=[101, 202],
        time_limits=[1000, 1500, 2000,2500],
        output_csv="time_limit.csv",
    )

if __name__ == "__main__":
    main()
```

Declaration of Authorship

I hereby declare that I have written this thesis independently and have not used any sources or aids other than those indicated. All passages taken either verbatim or in substance from other sources have been clearly identified as such. Furthermore, I declare that this thesis has not been submitted, in the same or a similar form, to any other examination authority.

<Place, Date (actual submission date, cf. title page)> <Signature>